

A Few Surprising Programming Language Features

The first programming language that you learn leaves an indelible mark on you. If you are used to coding in a particular language and then decide to take up a new one, the latter may throw up a few surprises. Some of these can be fun but they could also be frustrating.



I learned C more than 20 years ago and as a C programmer at heart, I found it difficult to adjust to other languages. Nevertheless, I had to learn or use many others like C++, Java, x86 assembly language, Python, etc, over the years. I am no expert in all these programming languages. If I can order food, ask for water and call a taxi in some (natural) language, I assume proficiency in that language. Often, the same standards apply while claiming proficiency in a programming language. Recently, while learning Haskell, a functional programming language, I came across a feature called lazy evaluation of expressions (to be

discussed later in this article), and I was literally surprised. This is something that has happened to me a lot while learning new programming languages. While learning Python, it was a surprise to learn that you don't need to explicitly declare the type of a variable before storing data into it.

This article discusses some of the unique features of different programming languages that might surprise programmers well versed in some other programming language. Now let us also try to find out why this occurs. Major surprises tend to occur when a programmer learns a language that has a different paradigm. For

example, a Java (an object-oriented programming language) programmer learning Haskell (a functional programming language) might come across a large number of features that are surprising and a few which might even look counter intuitive. Similarly, compiled and interpreted languages often differ a lot in their features. An example for this is a C++ (a compiled language) programmer learning Python (an interpreted language).

Similar surprises might await a person switching from a general-purpose programming language to a domain-specific one — for example, a C (a general-purpose

